# Time-Off Microservice - Technical Requirements Document

## 1. Executive Summary

The Time-Off Microservice is a critical component that bridges ReadyOn's internal time-off request management system with the customer's Human Capital Management (HCM) system (e.g., Workday, SAP). The service must maintain authoritative balance integrity by treating the HCM as the source of truth while providing employees and managers with real-time visibility into available leave balances.

**Key Objective**: Enable employees to request time off with confidence that their balance data is accurate, synchronized, and cannot be corrupted by timing issues, concurrent updates, or HCM-side changes.

---

## 2. Problem Statement

### 2.1 Core Challenge: Balance Synchronization

- Balances exist in two systems: ReadyOn and HCM
- HCM is the authoritative source but updates can occur outside ReadyOn's knowledge
- Examples of independent HCM updates:
  - Work anniversary bonuses
  - End-of-year/annual refreshes
  - Corrective adjustments by HR
  - Leave expiration policies
  - Company-wide policy changes

### 2.2 Race Condition Scenarios

1. **Employee requests leave** → Balance checked as 5 days, request submitted
2. **Simultaneously, HCM updates balance** → Now only 3 days available
3. **ReadyOn approval attempts HCM submission** → HCM rejects due to insufficient balance
4. **ReadyOn left in inconsistent state** (request marked approved but HCM rejected)

### 2.3 Data Dimension Complexity

- Balances are not simple single values
- Dimensions: `employeeId`, `locationId`, `leaveType`, `year` (or similar)
- HCM API accepts/expects proper dimension combinations
- Invalid dimension combinations should be rejected early

### 2.4 Defensive Requirements

- HCM errors are not guaranteed to catch all issues
- ReadyOn must validate balance sufficiency independently
- Reconciliation mechanisms for detecting drift
- Audit trail for all balance changes

---

# 3. System Scope

## 3.1 Primary Responsibilities

1. **Request Lifecycle Management**
   - Create, update, approve, reject, and cancel time-off requests
   - State transitions with validation at each step
2. **Balance Management**
   - Fetch and cache balance data from HCM
   - Sync full balance corpus periodically or on-demand
   - Maintain local balance state in SQLite
3. **Balance Validation**
   - Pre-approval validation against local cached balances
   - Synchronous balance sufficiency checks
   - Dimension validation (employee + location + leave type combinations)
4. **Synchronization**
   - Real-time balance sync before requests
   - Periodic batch syncs to detect independent HCM changes
   - Conflict resolution when drift is detected
   - Audit logging for reconciliation

## 3.2 Out of Scope

- Authentication/authorization (assume provided by API gateway)
- Employee self-service UI (this is the API layer)
- Manager approval workflow automation (manager interaction is REST API)
- HCM system design (we only consume its APIs)

---

# 4. Architecture Overview

## 4.1 System Components

```
┌──────────────────────────────────────────────────────────────────────┐
│                        ReadyOn Time-Off API                          │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │            NestJS Application (Single Service)              |    |
│  ├──────────────────────────────────────────────────────────────┤    │
│  | REST/GraphQL Layer                                           |    |
│  |  - Request endpoints (POST /requests, PATCH /requests/:id)|    |
│  |  - Balance endpoints (GET /balances/:employeeId/:location) |    |
│  |  - Admin endpoints (POST /sync, DELETE /cache)              |    |
│  └──────────────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  | Service Layer                                                |    |
│  |  - TimeOffRequestService (request lifecycle)                 |    |
│  |  - BalanceService (fetch, validate, cache)                   |    |
│  |  - SyncService (batch and real-time syncing)                 |    |
│  |  - ConflictResolutionService (drift detection)               |    |
│  |  - AuditService (logging and reconciliation)                 |    |
│  └──────────────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  | Data Layer (SQLite)                                          |    |
│  |  - requests table                                            |    |
│  |  - balances table (cached from HCM)                          |    |
│  |  - balance_history table (drift detection)                   |    |
│  |  - audit_logs table (traceability)                           |    |
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘

          ↕ (REST/HTTP)              ↕ (REST/HTTP)

    ┌──────────────────┐          ┌──────────────────┐
    |  HCM Real-Time   |          |  HCM Batch       |
    |  API             |          |  Sync Endpoint   |
    |  (Per-request)   |          |  (Full corpus)   |
    └──────────────────┘          └──────────────────┘
```

## 4.2 Data Model

### Requests Table

```
id (UUID)
employee_id (String)
location_id (String)
leave_type (String)
start_date (Date)
end_date (Date)
days_requested (Decimal)
status (PENDING, APPROVED, REJECTED, CANCELLED)
balance_snapshot (JSON) - cached balance at request time
submitted_at (Timestamp)
approved_at (Timestamp, nullable)
rejection_reason (String, nullable)
hcm_sync_status (SYNCED, PENDING, FAILED)
hcm_request_id (String, nullable)
created_at (Timestamp)
updated_at (Timestamp)
```

### Balances Table (Cached from HCM)

```
id (UUID)
employee_id (String)
location_id (String)
leave_type (String)
available_balance (Decimal)
used_balance (Decimal)
carryover_balance (Decimal, nullable)
balance_year (Integer)
hcm_last_updated_at (Timestamp)
readyon_cached_at (Timestamp)
is_stale (Boolean) - indicates drift from HCM
created_at (Timestamp)
updated_at (Timestamp)
```

### Balance History Table (For Drift Detection)

```
id (UUID)
employee_id (String)
location_id (String)
leave_type (String)
previous_balance (Decimal)
new_balance (Decimal)
change_source (HCM_SYNC, REQUEST_APPROVAL, REQUEST_REJECTION, ADMIN_ADJUSTMENT)
detected_at (Timestamp)
hcm_source_timestamp (Timestamp, nullable)
is_drift (Boolean) - true if change not initiated by ReadyOn
created_at (Timestamp)
```

### Audit Logs Table

```
id (UUID)
entity_type (REQUEST, BALANCE, SYNC)
entity_id (String)
action (CREATE, UPDATE, DELETE, SYNC, CONFLICT_DETECTED)
old_value (JSON)
new_value (JSON)
reason (String)
performed_by (String - system or user ID)
ip_address (String, nullable)
created_at (Timestamp)
```

---

# 5. Key Challenges & Solutions

## Challenge 1: Race Condition During Request Approval

**Problem**:

- Employee sees 5 days available, submits request for 2 days
- HCM updates balance to 3 days before approval processing
- Approval fails in HCM, but request already marked approved locally

**Solution**:

1. **Pessimistic Locking**: Lock balance row during approval transaction
2. **Pre-Approval Sync**: Always sync balance from HCM before approval (not before submission)
3. **Atomic Transaction**: Combine balance check + HCM request submission in single transaction
4. **Retry with Backoff**: If HCM sync fails, retry with exponential backoff (3 attempts)

5. **Transaction Rollback**: If HCM approval fails, rollback local approval immediately
6. **State Machine**: Request states enforce valid transitions (can't approve if balance insufficient at approval time)

## Challenge 2: Detecting Independent HCM Balance Changes

**Problem**:

- ReadyOn caches balance = 10 days
- Work anniversary: HCM updates balance to 15 days
- ReadyOn is unaware, thinks balance is still 10
- Employee makes request, ReadyOn's approval succeeds, but HCM has extra 5 days unused

**Solution**:

1. **Periodic Batch Sync**: Run sync every 6 hours (configurable) to compare local vs HCM
2. **Change Detection**: Compare `hcm_last_updated_at` timestamp with our `readyon_cached_at`
3. **Drift Alerts**: Flag balance as stale if HCM timestamp > ReadyOn timestamp
4. **Reconciliation Logic**:
   - If balance increased: Update local cache, no action needed (employee has more leave)
   - If balance decreased: Flag for review, notify managers
   - If dimensions changed: Invalidate and resync entire employee

## Challenge 3: Dimension Validation

**Problem**:

- Invalid dimension combo: employeeId "E123" at locationId "NYC" with leaveType "SICK" doesn't exist
- HCM may or may not validate this
- ReadyOn accepts request → HCM rejects → inconsistent state

**Solution**:

1. **Early Validation**: Validate dimension combo at request creation time
2. **Dimension Registry**: Maintain list of valid employee-location-leaveType combinations
3. **Sync Dimensions**: During batch sync, validate that local dimensions match HCM
4. **Request Rejection**: If dimension invalid, reject with clear error message
5. **Audit Trail**: Log all dimension validation attempts for debugging

## Challenge 4: Handling HCM Errors

**Problem**:

- HCM API timeout during approval
- HCM returns ambiguous error message
- HCM is partially down (some endpoints work, others don't)

**Solution**:

1. **Circuit Breaker**: Stop attempting HCM calls if too many failures
2. **Graceful Degradation**: Queue requests for async processing if HCM is down
3. **Error Classification**:
   - Permanent failures (invalid dimension) → Reject immediately
   - Temporary failures (timeout) → Retry with backoff
   - Insufficient balance → Reject with clear message
4. **Health Check Endpoint**: Monitor HCM connectivity separately
5. **Fallback**: If HCM down but balance was recently synced, allow requests based on cached balance (with warning)

## Challenge 5: Concurrent Request Approval

**Problem**:

- Two managers approve requests simultaneously for same employee
- Balance = 10 days, both requests for 6 days
- Both approvals think they're within balance

**Solution**:

1. **Optimistic Locking**: Use version numbers on balance rows
2. **Serialization**: Process approvals sequentially per employee per location (queue-based)
3. **Balance Holds**: Mark balance as "reserved" pending approval confirmation
4. **Conflict Resolution**: If approval fails due to conflict, queue for manager review

---

# 6. API Specification

## 6.1 Request Management Endpoints

### Create Time-Off Request

```
POST /api/v1/requests
Content-Type: application/json

{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "startDate": "2026-05-01",
  "endDate": "2026-05-03",
  "daysRequested": 3,
  "reason": "Family vacation"
}

Response: 201 Created
{
  "id": "REQ-12345",
  "status": "PENDING",
  "currentBalance": 7,
  "balanceAfter": 4,
  "balanceSnapshotAt": "2026-04-24T10:30:00Z"
}
```

### Approve Time-Off Request

```
PATCH /api/v1/requests/{requestId}/approve
Content-Type: application/json

{
  "approvedBy": "MGR-001"
}

Response: 200 OK
{
  "id": "REQ-12345",
  "status": "APPROVED",
  "approvedAt": "2026-04-24T10:35:00Z",
  "hcmSyncStatus": "SYNCED",
  "hcmRequestId": "HCM-98765"
}

Possible Error Responses:
- 409 Conflict: Balance insufficient (drift detected)
- 503 Service Unavailable: HCM down, cannot proceed
- 400 Bad Request: Invalid state transition
```

### Reject Time-Off Request

```
PATCH /api/v1/requests/{requestId}/reject
Content-Type: application/json

{
  "rejectionReason": "Coverage requirements not met"
}

Response: 200 OK
```

### Cancel Time-Off Request

```
PATCH /api/v1/requests/{requestId}/cancel
Content-Type: application/json

{
  "cancellationReason": "Changed my mind"
}

Response: 200 OK
```

### Get Request Details

```
GET /api/v1/requests/{requestId}

Response: 200 OK
{
  "id": "REQ-12345",
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "startDate": "2026-05-01",
  "endDate": "2026-05-03",
  "daysRequested": 3,
  "status": "APPROVED",
  "balanceSnapshot": { ... },
  "createdAt": "2026-04-24T10:30:00Z",
  "approvedAt": "2026-04-24T10:35:00Z"
}
```

## 6.2 Balance Management Endpoints

### Get Current Balance

```
GET /api/v1/balances/{employeeId}/{locationId}?leaveType=VACATION

Response: 200 OK
{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "availableBalance": 7,
  "usedBalance": 3,
  "carryoverBalance": 2,
  "balanceYear": 2026,
  "lastSyncedAt": "2026-04-24T10:00:00Z",
  "isStale": false
}
```

### Get All Balances for Employee

```
GET /api/v1/balances/{employeeId}

Response: 200 OK
{
  "employeeId": "EMP-001",
  "balances": [
    {
      "locationId": "LOC-NYC",
      "leaveType": "VACATION",
      "availableBalance": 7,
      ...
    },
    {
      "locationId": "LOC-NYC",
      "leaveType": "SICK",
      "availableBalance": 10,
      ...
    }
  ]
}
```

## 6.3 Synchronization Endpoints

### Sync Balances from HCM (Batch)

```
POST /api/v1/admin/sync/balances
Content-Type: application/json

{
  "employeeIds": ["EMP-001", "EMP-002"], // optional, all if omitted
  "fullSync": false // if true, reset all local data first
}

Response: 202 Accepted
{
  "syncId": "SYNC-001",
  "status": "IN_PROGRESS",
  "startedAt": "2026-04-24T10:30:00Z"
}
```

### Get Sync Status

```
GET /api/v1/admin/sync/{syncId}

Response: 200 OK
{
  "syncId": "SYNC-001",
  "status": "COMPLETED",
  "totalRecords": 150,
  "successCount": 148,
  "errorCount": 2,
  "errors": [
    { "employeeId": "EMP-999", "reason": "Employee not found in HCM" }
  ],
  "completedAt": "2026-04-24T10:35:00Z"
}
```

### Detect Balance Drift

```
POST /api/v1/admin/detect-drift

Response: 200 OK
{
  "driftDetected": true,
  "driftedBalances": [
    {
      "employeeId": "EMP-001",
      "locationId": "LOC-NYC",
      "leaveType": "VACATION",
      "previousBalance": 10,
      "currentBalance": 15,
      "changeDetectedAt": "2026-04-24T10:30:00Z"
    }
  ]
}
```

---

# 7. Data Synchronization Strategy

## 7.1 Real-Time Sync (Before Approval)

```
Flow:
1. Manager clicks "Approve" on request
2. System fetches current balance from HCM real-time API
3. If balance sufficient:
   a. Lock balance row in SQLite
   b. Submit time-off to HCM
   c. If HCM succeeds: Update local balance, mark request APPROVED, release lock
   d. If HCM fails: Rollback, keep request PENDING, release lock
4. If balance insufficient: Reject request, release lock

Timeout: 5 seconds max for HCM call
Retry: 3 attempts with exponential backoff
```

## 7.2 Batch Sync (Periodic)

```
Flow (every 6 hours by default):
1. Get all active employees from ReadyOn
2. For each employee:
   a. Call HCM batch endpoint for full balance corpus
   b. Compare returned balances with local cache
   c. For each balance:
      - If HCM balance > local: Update cache (employee gained time)
      - If HCM balance < local: Alert HR, flag as drift (unexpected loss)
      - If dimensions differ: Resync entire employee
3. Log all changes in balance_history table
4. Trigger notifications if significant drift detected

Timeout: 30 seconds per employee
Parallelism: Process 5 employees concurrently
```

### 7.3 On-Demand Sync

```
Triggered by:
- Employee manual refresh button
- Manager viewing balance before approval
- Admin trigger via API
- After conflict detection

Response Time: < 2 seconds
```

---

## 8. Error Handling & Resilience

### 8.1 HCM Availability States

| State | ReadyOn Behavior | Risk |
|---|---|---|
| Healthy | Real-time balance checks, immediate approval | None |
| Slow (5s timeout) | Retry with exponential backoff, queue async | Delayed approval |
| Intermittent (fails 50% of time) | Circuit breaker, reject requests | Requests rejected, but balance not lost |
| Down (no response) | Use cached balance, mark requests as "pending HCM sync" | Approval based on stale data |

### 8.2 Retry Policy

```
Request Approval Sync:
- Attempt 1: Immediate
- Attempt 2: After 500ms
- Attempt 3: After 1500ms
- Fail: Return 503 Service Unavailable

Batch Sync:
- Attempt 1: Immediate
- Attempt 2: After 2 seconds
- Attempt 3: After 5 seconds
- Continue in async queue if fails
```

### 8.3 Circuit Breaker

```
- Threshold: 5 consecutive HCM failures in 2 minutes
- Action: Stop attempting real-time requests, queue for async
- Recovery: Try again every 30 seconds until successful
- Fallback: Use cached data (within 24 hours) with warnings
```

---

## 9. Security & Compliance

### 9.1 Data Validation

- All inputs validated against schema before processing
- SQL injection prevention via parameterized queries (TypeORM)
- JSON schema validation for HCM responses

### 9.2 Audit Trail

- Every balance change logged with:
  - Old and new value
  - Change source (HCM, request approval, admin)
  - Timestamp
  - User/system that initiated change
- Request lifecycle fully audited
- Cannot delete historical data (append-only balances/audit logs)

### 9.3 Access Control

- Employees can only view/request their own time off
- Managers can view/approve only direct reports
- Admins can force sync, view audit logs
- API assumes authentication layer enforces these roles

---

# 10. Performance Considerations

## 10.1 Database Indexing

```
-- Critical indexes
CREATE INDEX idx_requests_employee_status ON requests(employee_id, status);
CREATE INDEX idx_requests_created_at ON requests(created_at DESC);
CREATE INDEX idx_balances_employee_location ON balances(employee_id, location_id,
leave_type);
CREATE INDEX idx_balance_history_employee_date ON balance_history(employee_id,
created_at DESC);
CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type, entity_id);
```

## 10.2 Caching Strategy

- Balance cache TTL: 30 minutes (refresh on read if stale)
- Request list TTL: 5 minutes
- Dimension registry TTL: 1 hour
- Invalidate cache on:
  - Successful HCM sync
  - Approval/rejection of request
  - Manual cache clear by admin

## 10.3 Scalability

- Designed for up to 10,000 concurrent requests
- Batch operations parallelized up to 10 threads
- Queue-based approval processing if surge detected
- SQLite suitable for single-region deployment

---

# 11. Testing Strategy

## 11.1 Unit Tests (40% coverage)

- Service layer logic (balance validation, state transitions)

- Error handling and retry mechanisms
- Data transformation and mapping

### 11.2 Integration Tests (45% coverage)

- API endpoint validation
- Database transaction handling
- Balance sync scenarios

### 11.3 End-to-End Tests (15% coverage)

- Full request lifecycle with mock HCM
- Race condition scenarios
- Drift detection and reconciliation
- Error recovery and fallback scenarios

### 11.4 Load & Stress Tests

- 1000 concurrent requests/minute
- HCM latency simulation (500ms-5s delays)
- HCM failure injection
- Verify no data loss or corruption

---

## 12. Implementation Timeline

| Phase | Duration | Deliverables |
|---|---|---|
| Phase 1: Schema & Mock HCM | 1 week | SQLite schema, Mock HCM endpoints |
| Phase 2: Core Services | 2 weeks | Balance & Request services, real-time sync |
| Phase 3: API Layer | 1 week | REST endpoints, validation |
| Phase 4: Sync & Resilience | 1 week | Batch sync, circuit breaker, retry logic |
| Phase 5: Testing & Deployment | 1.5 weeks | Full test suite, documentation |

---

## 13. Alternatives Considered

### Alternative 1: Eventual Consistency Model

**Approach**: Don't block on HCM sync; queue all approvals and process async

**Pros**:

- Faster user-facing API
- No need for timeout/retry logic
- Simpler error handling

**Cons**:

- Approval confirmations delayed (not real-time)
- Manager must check status later
- Risk of approving when balance insufficient (worse UX)
- Harder to explain failed approvals after confirmation

**Decision**: REJECTED - Immediate feedback required for good UX

---

## Alternative 2: Full Local Balance Ownership

**Approach**: ReadyOn decrements balance locally, syncs with HCM async, reconciles later

**Pros**:

- Fastest response times (no HCM call needed)
- Simpler logic

**Cons**:

- Race conditions with other HCM clients harder to detect
- Large discrepancies possible before detection
- No real-time safety guarantee
- Violates "HCM is source of truth" requirement

**Decision**: REJECTED - Compromises data integrity

---

## Alternative 3: Read-Only Cache, All Writes Async

**Approach**: ReadyOn never writes to HCM synchronously; only caches reads

**Pros**:

- No transactional complexity
- Guaranteed HCM consistency

**Cons**:

- Managers get no confirmation of approval success
- Approval UI must show "pending" for hours
- Complex async reconciliation needed
- Bad user experience

**Decision**: REJECTED - Poor UX, too complex

---

### Alternative 4: Pessimistic Locking (Selected)

**Approach**: Lock balance row when request submitted, hold until approval/rejection

**Pros**:

- Prevents race conditions entirely
- Simple, predictable behavior
- Strong consistency guarantees

**Cons**:

- Lock contention if many concurrent requests
- Longer transaction times
- Deadlock potential

**Mitigation**:

- Lock timeout of 30 seconds
- Deadlock detection and retry
- Async queue if lock wait exceeds 10 seconds

**Decision**: SELECTED - Best balance of safety and performance

---

## 14. Success Criteria

1. ✅ No data corruption or loss under concurrent operations
2. ✅ Balance discrepancies detected within 6 hours
3. ✅ Request approval time < 5 seconds (95th percentile) when HCM healthy
4. ✅ All balance changes auditable with full history
5. ✅ System continues operating even if HCM unavailable > 2 hours
6. ✅ Test coverage > 80% with no critical paths untested
7. ✅ All state transitions validated and prevented if invalid
8. ✅ Dimension mismatches caught before HCM submission

---

## 15. Conclusion

The Time-Off Microservice requires careful attention to concurrency, data consistency, and HCM integration. By implementing pessimistic locking, real-time pre-approval syncs, periodic batch

reconciliation, and comprehensive auditing, we can maintain balance integrity while providing employees and managers with confidence in the system.

The proposed architecture balances performance with safety, ensuring that no balance is lost and no over-approval occurs, while still providing sub-second response times for normal operations.